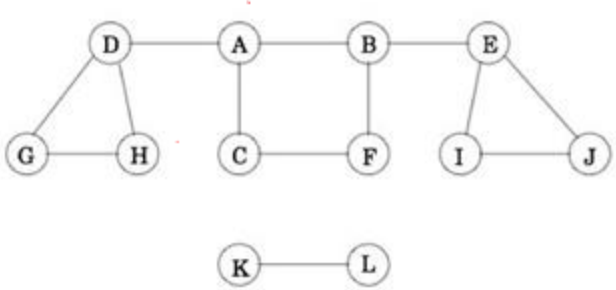
Graph Algorithms

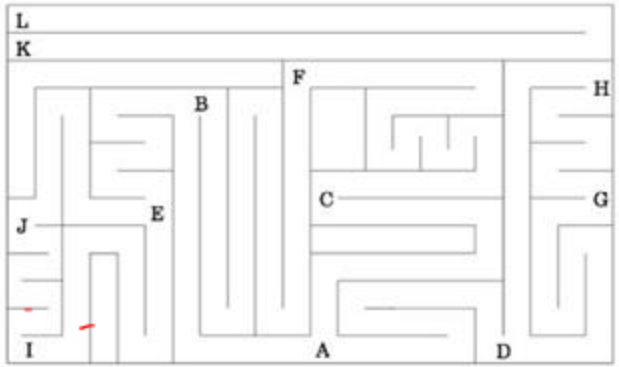
Depth-First Search (DFS)

Depth-First Search (DFS)

- **Input**: A graph G = (V, E) and a vertex s
- **Output**:
  - Explores all the nodes in the graph reachable from **s**
  - generates a tree rooted at **s**

DFS Application



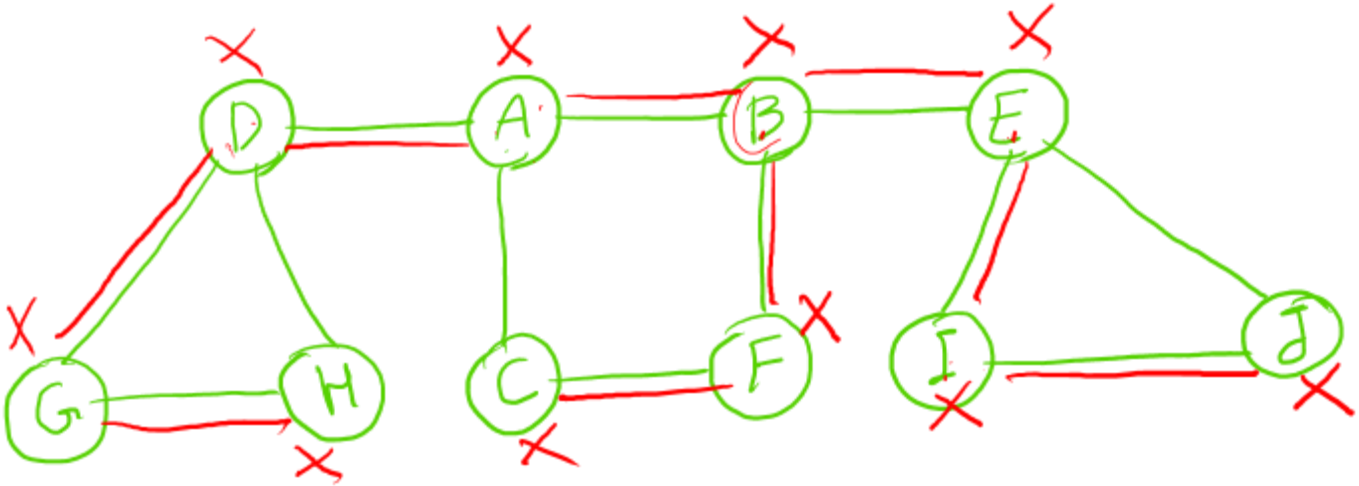


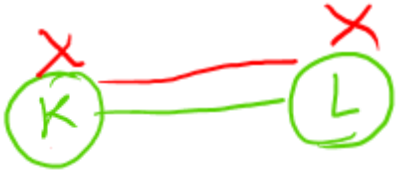
The picture is from textbook: http://algorithmics.lsi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf

DFS: Version 1

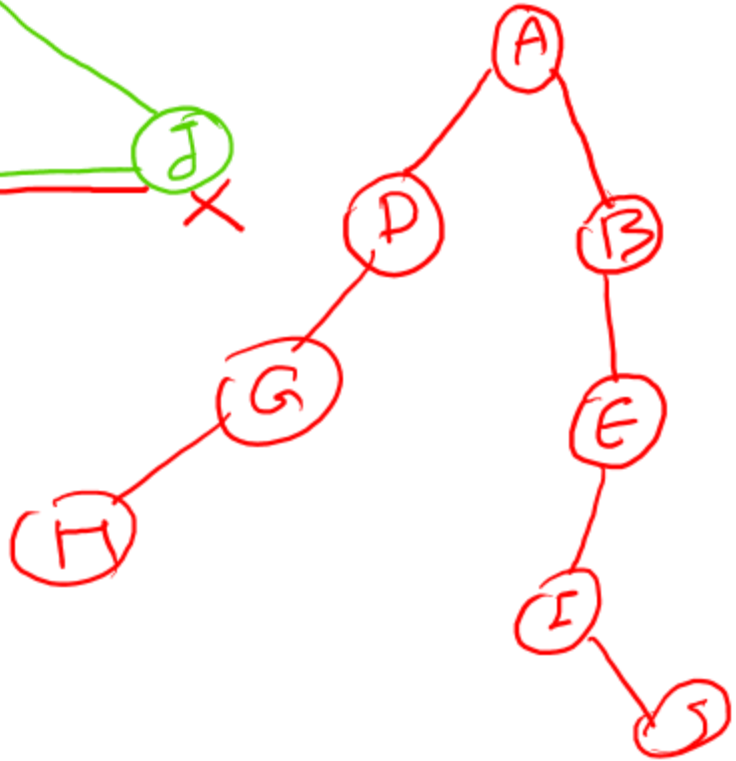
```
DFS(G)
    for each vertex u in G:
            visited[u] = Fale  #Mark u as undiscovered
    for each vertex u in G:
        if visited[u] == False  #if u is undiscovered
            DFS(G, u)
DFS(G, u)
    visited[u] = True   #discover u
    for each neighbor v of u:
         if (visited[v] == False) #if v is undiscovered
            DFS(G, v)
```

DFS Example:

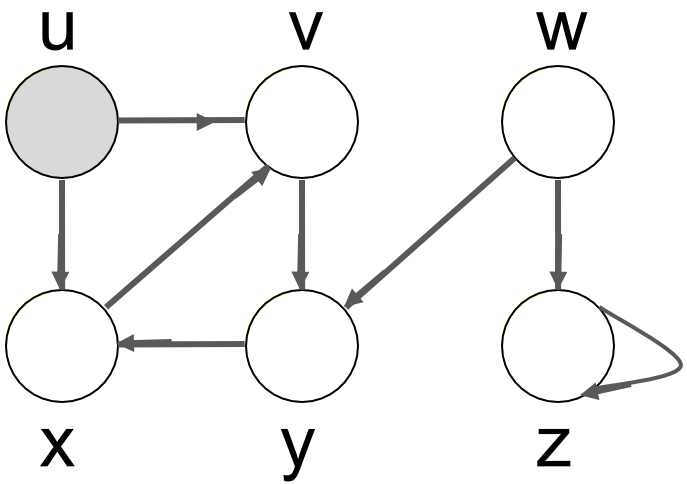




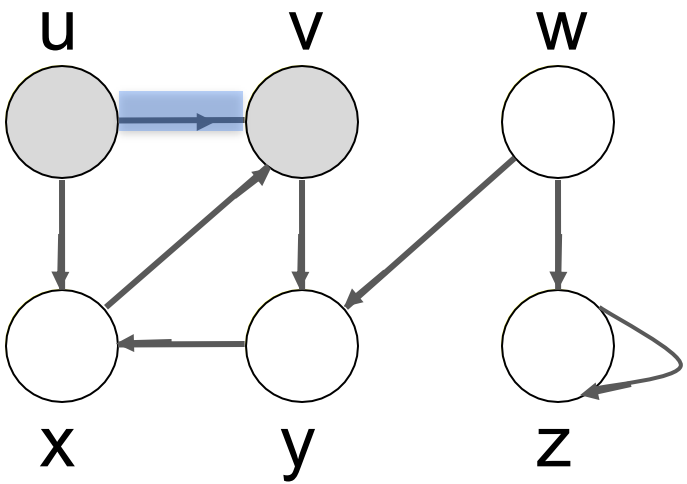




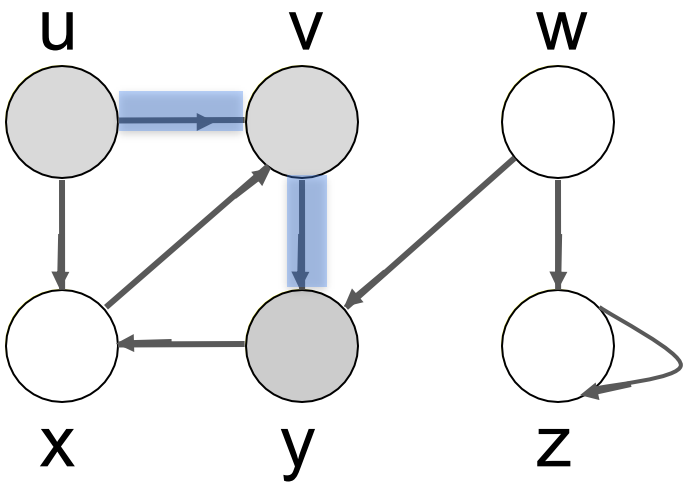
DFS Example



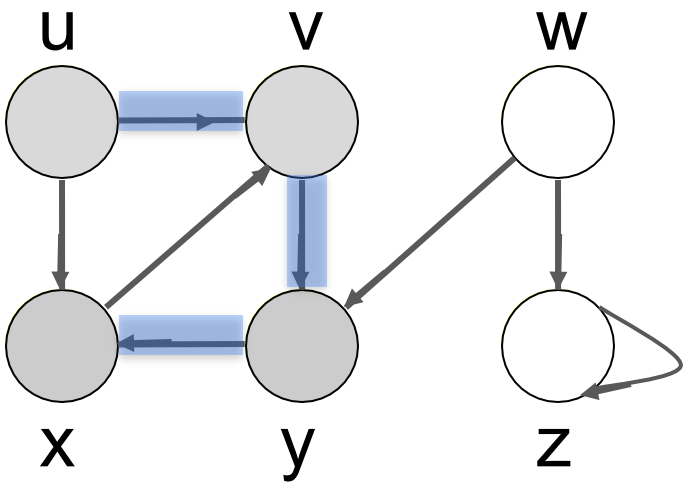
DFS Example



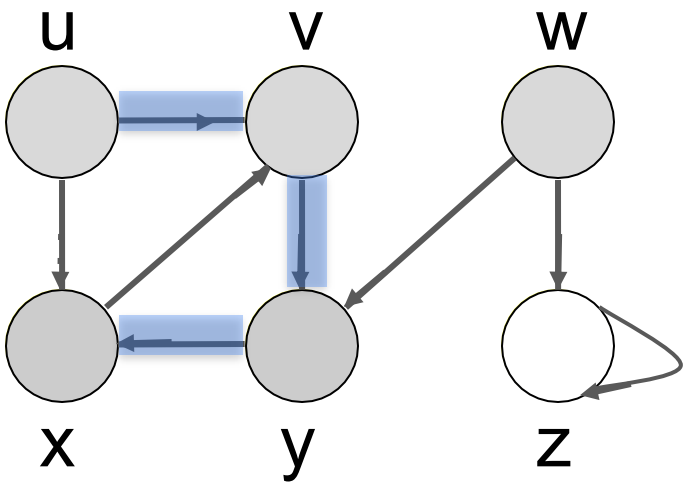
DFS Example



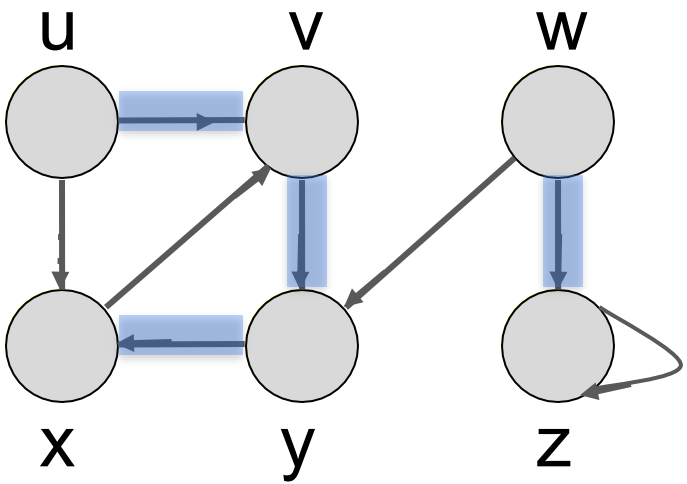
DFS Example



DFS Example



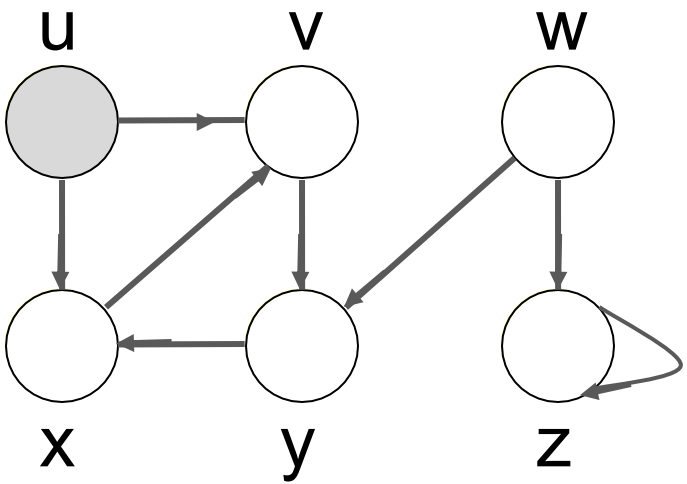
DFS Example



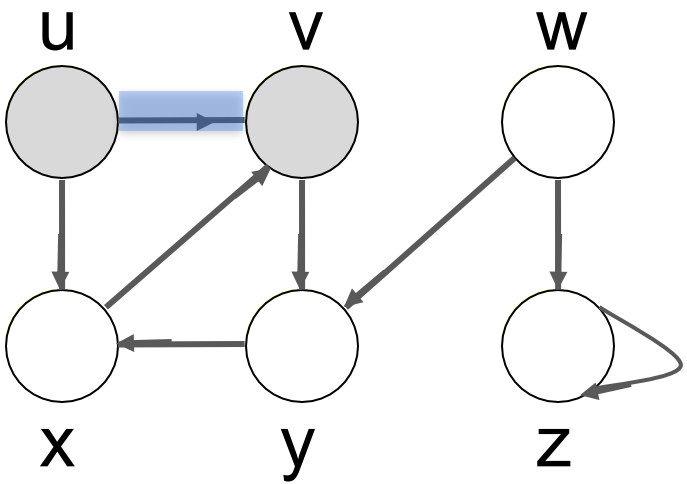
DFS: Version 2

```
DFS(G)
    for each vertex u in G:
        color[u] = WHITE  #Mark u as undiscovered
    for each vertex u in G:
        if color[u] == WHITE  #if u is undiscovered
            DFS(G, u)
DFS(G, u)
    color[u] = GRAY   #discover s
    for each neighbor v of u:
        if (color[v] == WHITE) #if v is undiscovered
            DFS(G, v)
    color[u] = BLACK   #Mark s as fully-discovered(BLACK)
```

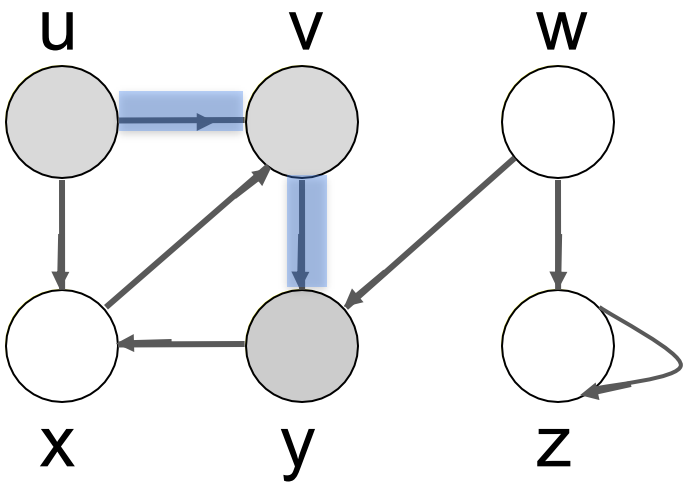
DFS Example



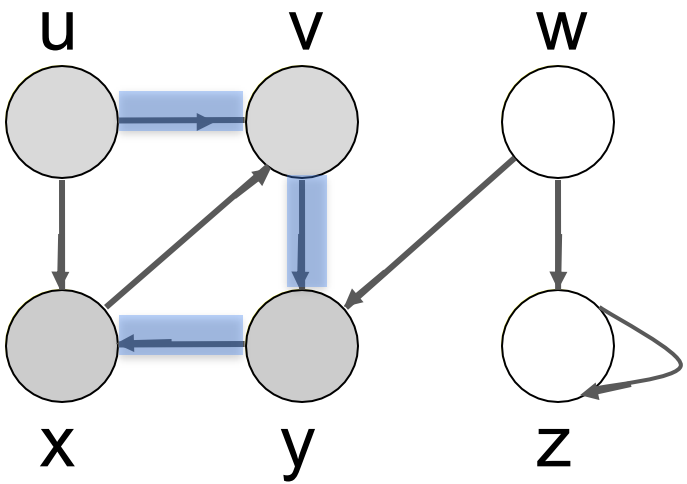
DFS Example



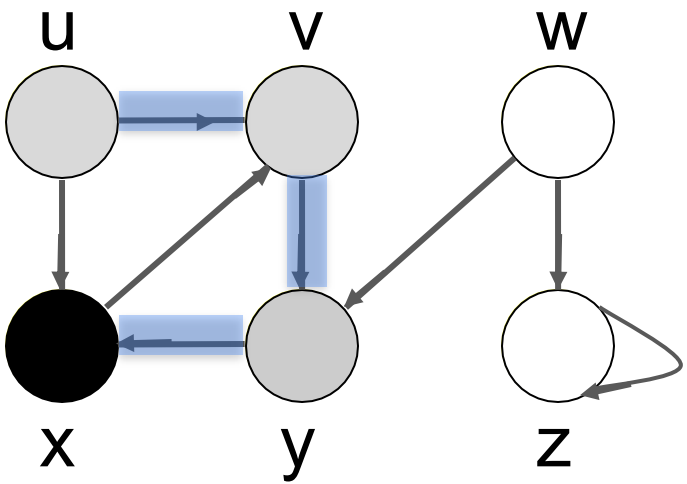
DFS Example



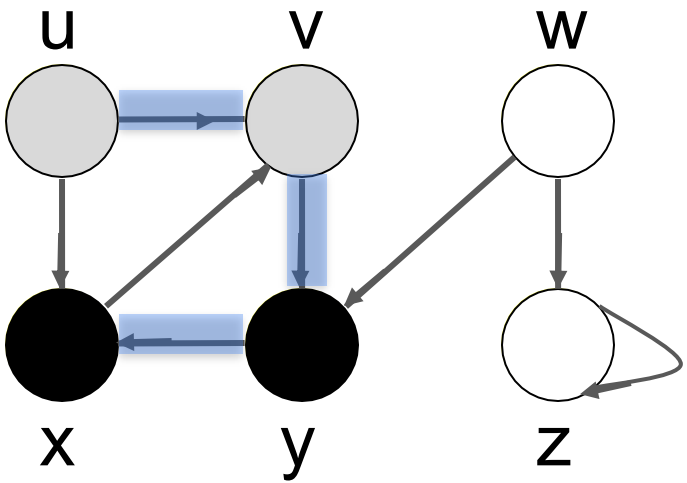
DFS Example



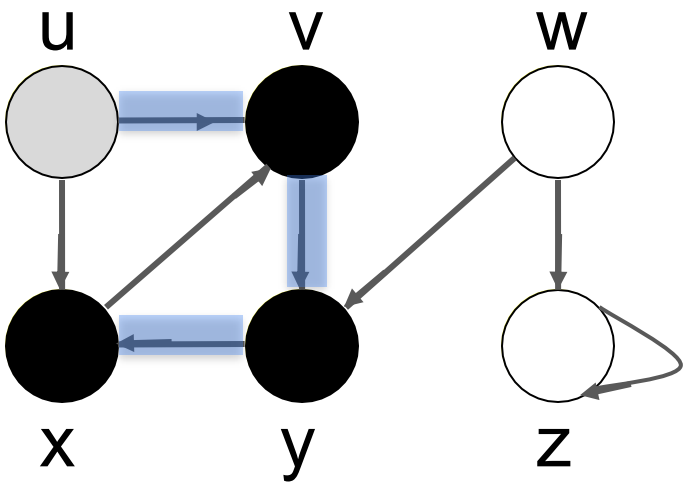
DFS Example



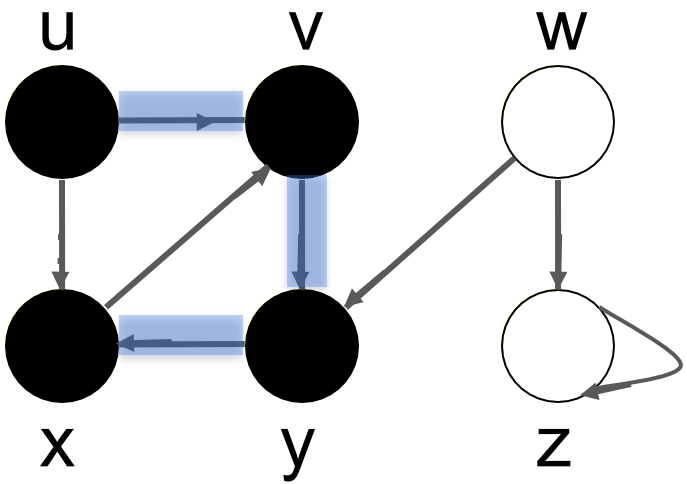
DFS Example



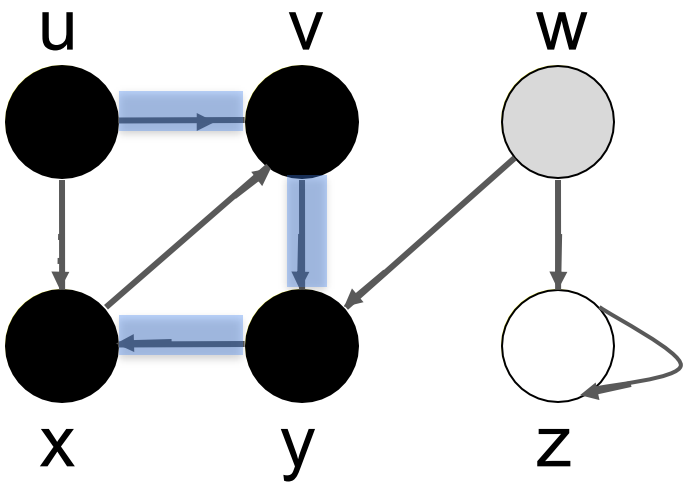
DFS Example



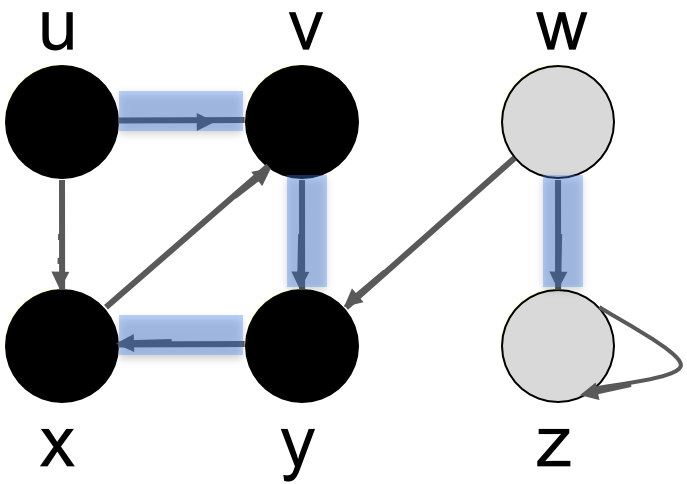
DFS Example



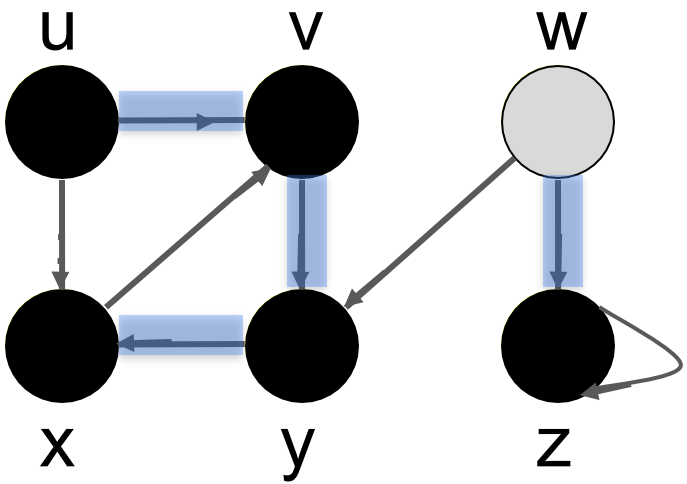
DFS Example



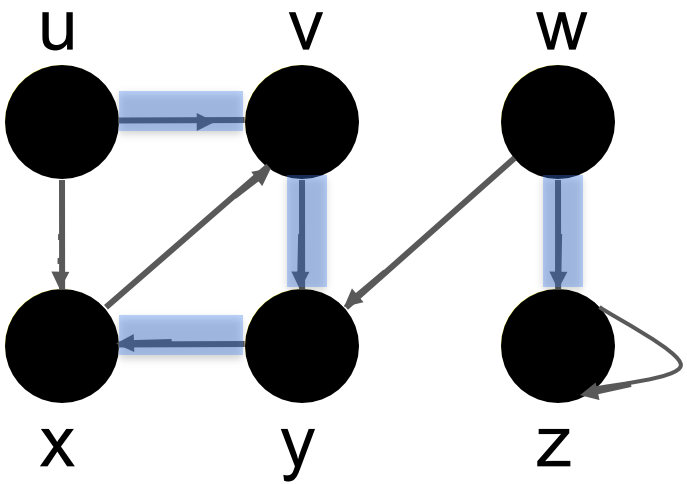
DFS Example



DFS Example



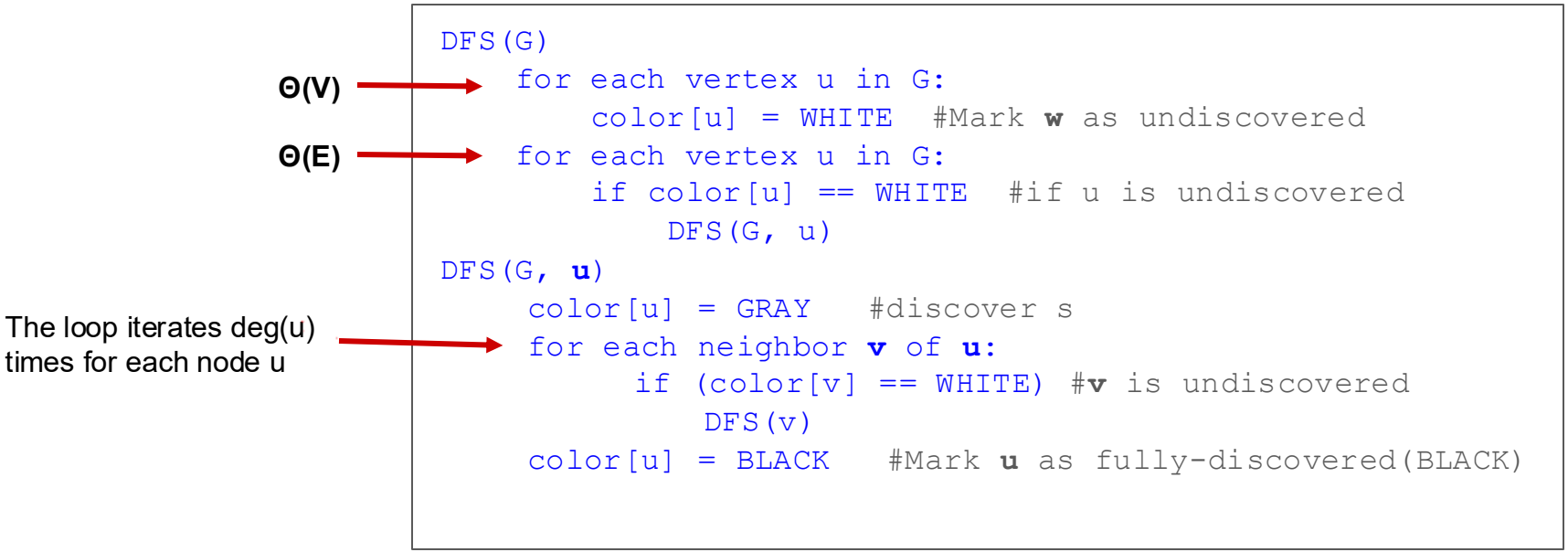
DFS Example



Depth-First Search (DFS)

- **Input**: A graph G = (V, E) and a vertex s
- **Output**:
  - Explores all the nodes in the graph reachable from **s**
  - generates a tree rooted at **s**
- Algorithm:
  - It starts from **s**, and follows the first path it finds and goes as deep as possible
  - During the execution of the algorithm, vertices are in one of the three following states:
    - UnDiscovered (white)
    - Discovered (gray)
    - Fully explored (black)
- Algorithm is implemented recursively
  - Can be implemented non-recursively using a stack

DFS: Runtime Analysis



Runtime: Θ(V + E) :
For each vertex u, the recursive function DFS(G, u) is called at most once.
When DFS(G, u) is called, the for loop is executed at most deg(u) times.

- In total the runtime of for loop is Θ (∑deg(u)) = Θ(E)

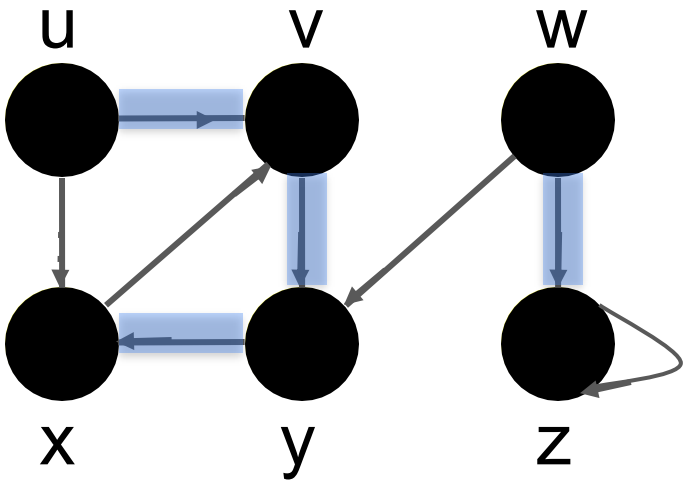
DFS Tree / Forest

- Similar to BFS, we can construct a DFS tree
  - It could be used to find the path from s to all the reachable vertice
- When a vertex v is first discovered when exploring vertex u, we say vertex u is the parent of vertex v
- The edges (v, parent(v)) form a tree, and we can use them to find a path to s.
- A graph could have many different DFS trees depending on the order of exploring the neighbors of vertices

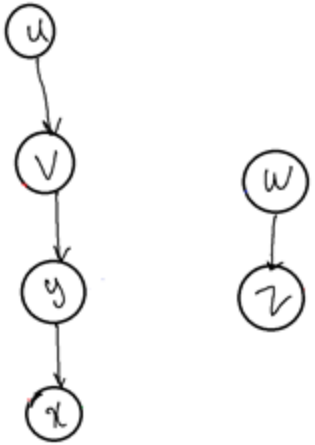
DFS: Keep track of the path

```
DFS(G)
    for each vertex u in G:
         color[u] = WHITE  #Mark w as undiscovered
        Pred[u] = NULL
    for each vertex u in G:
        if color[u] == WHITE  #if u is undiscovered
            DFS(G, u)
DFS(G, u)
     color[u] = GRAY   #discover s
     for each neighbor v of u:
          if (color[v] == WHITE) #v is undiscovered
              pred[v] = u
              DFS(v)
     color[u] = BLACK   #Mark u as fully-discovered(BLACK)
```

DFS Example



DFS Tree/Forest

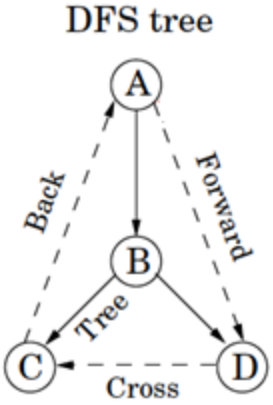


DFS: DFS(G, s)

Lemma 1. There is a path from **s** to **v** if and only if **v** is discovered by DFS(G, s)

DFS: classification of edges in the graph

- DFS classifies the edges of the input Graph G=(V, E).
  - It specifies the relationship between the nodes
- In a **directed** graph, an edge (u, v) is in one of the following classes:
  - Tree edge
    - If it is in the depth-first forest generated by the algorithm
  - Back edge
    - If **v** is ancestor of **u** in a depth-first tree
  - Forward edge
    - If **v** is a descendant of **u** in a depth-first tree
  - Cross edges
    - If it is not in any of the above category. They can go between vertices in the same depth-first tree, as long as one vertex is not an ancestor of the other, or they can go between vertices in different depth-first trees



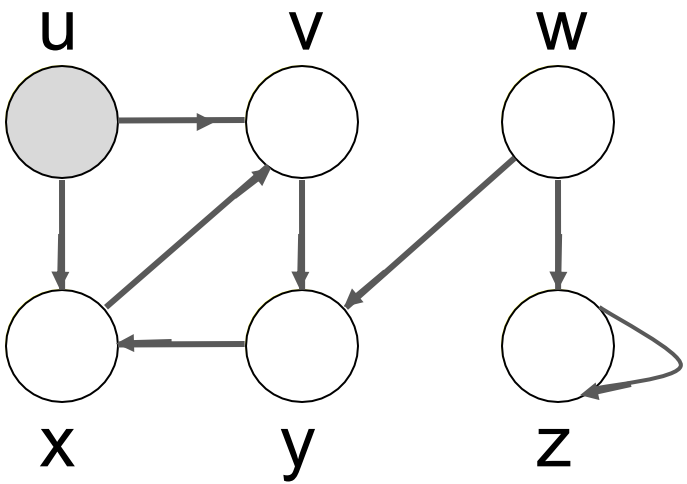
Question

How three colors (WHITE, GRAY, BLACK) helps in identifying different types of edges?

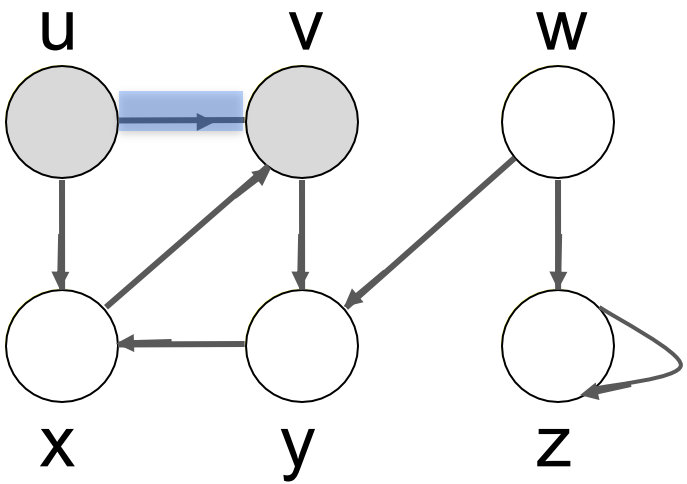
DFS: Classification of edges: directed graph

- When we first explore an edge (u, v), the color of vertex v determine edge class
- WHITE: a tree edge
- GRAY: a back edge
- BLACK: a forward or cross edge
  - ?????

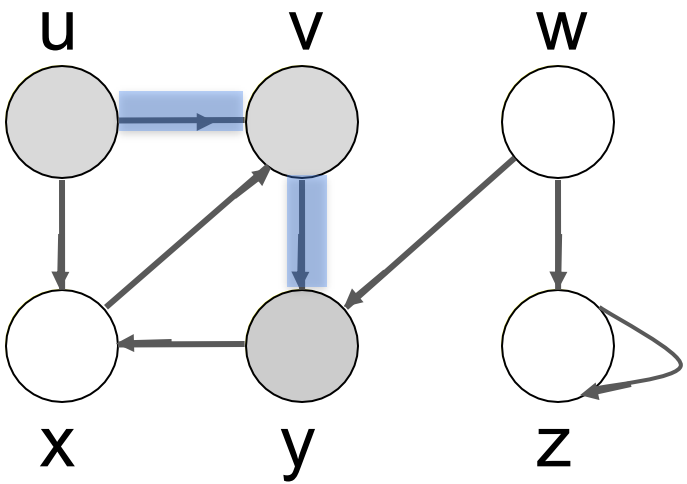
DFS Example



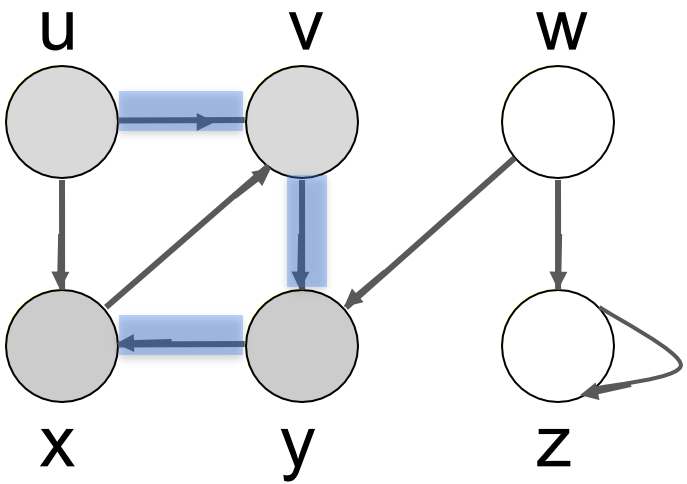
DFS Example



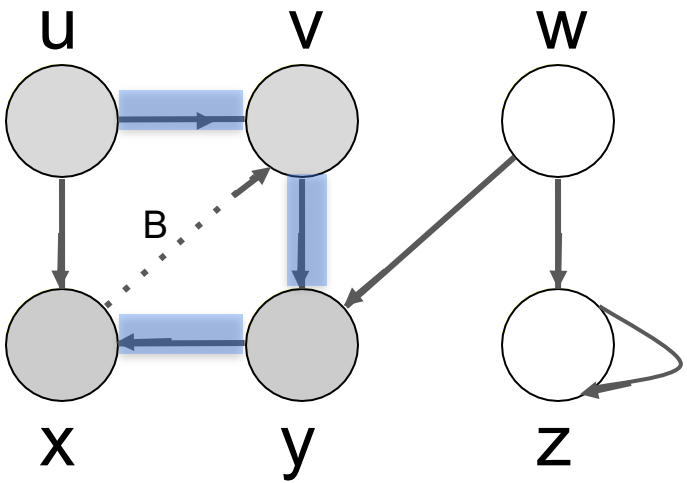
DFS Example



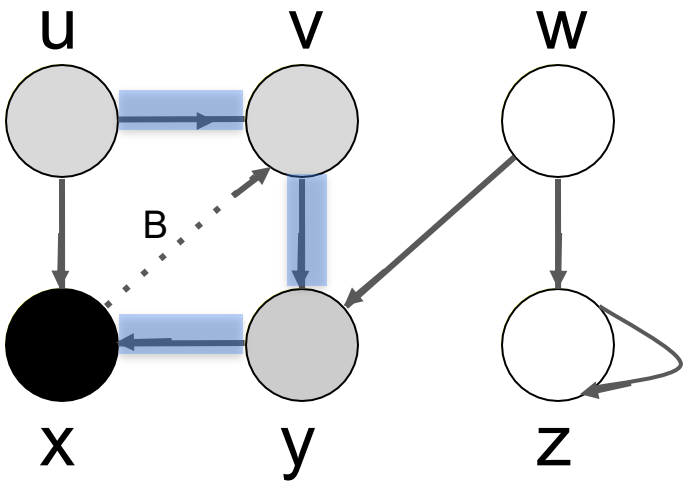
DFS Example



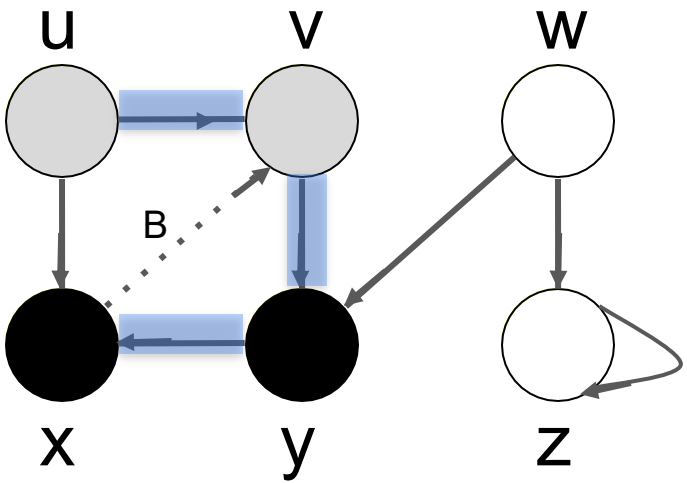
DFS Example



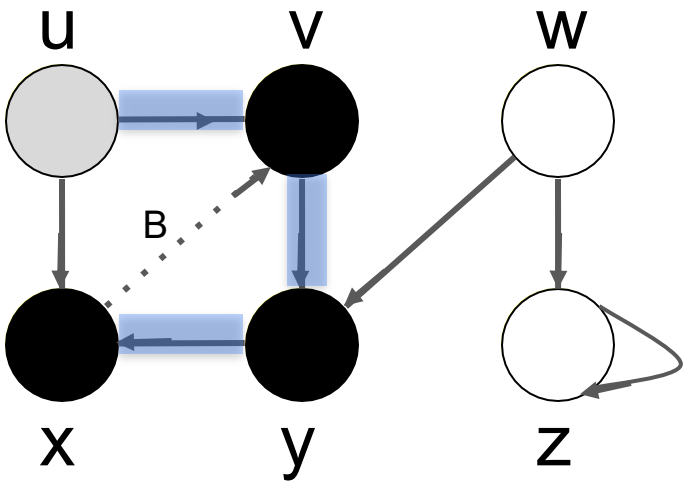
DFS Example



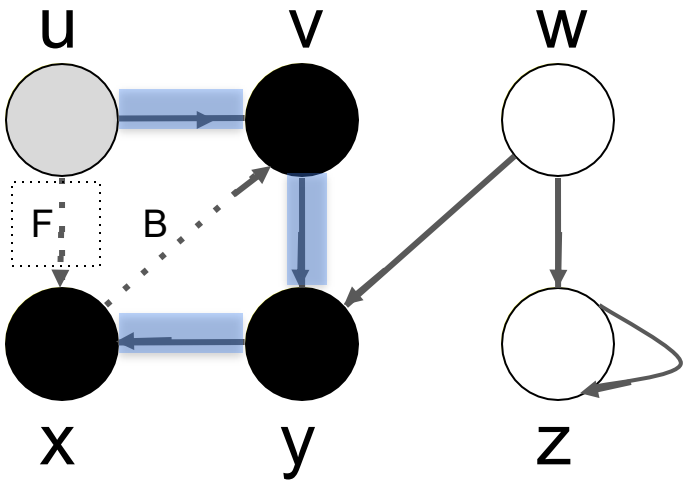
DFS Example



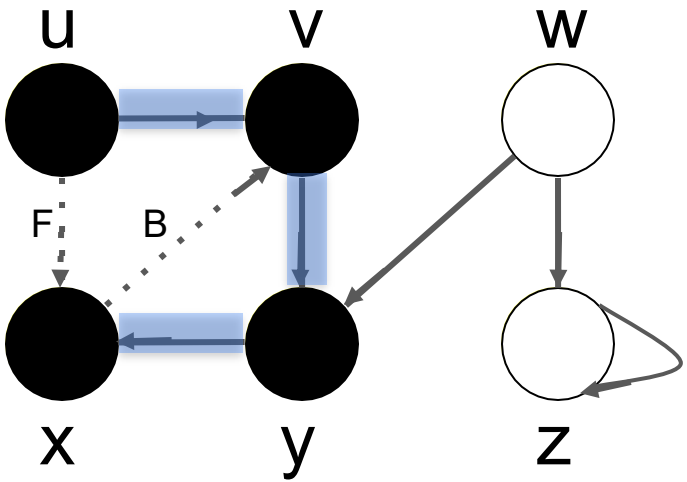
DFS Example



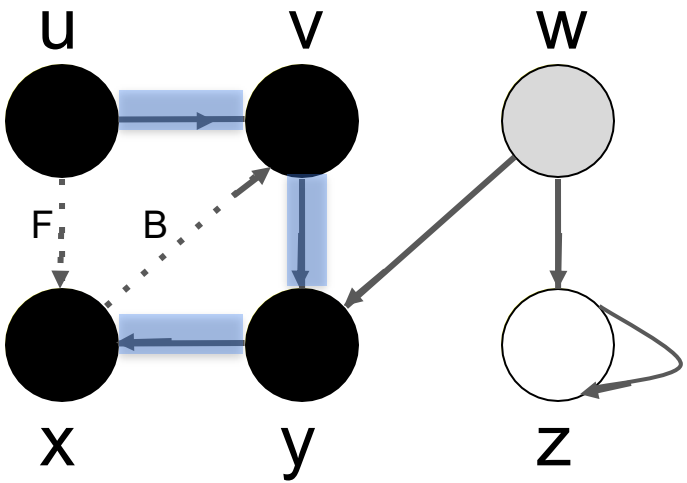
DFS Example



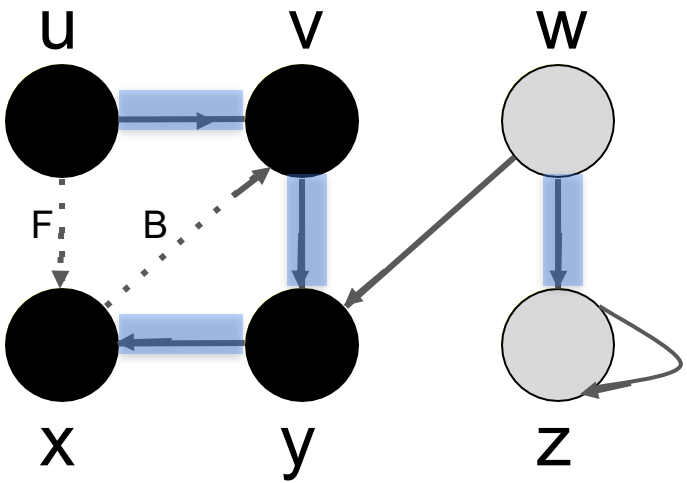
DFS Example



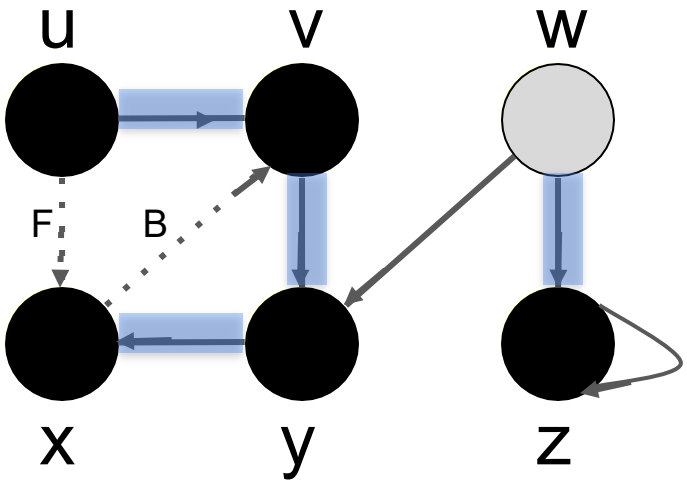
DFS Example



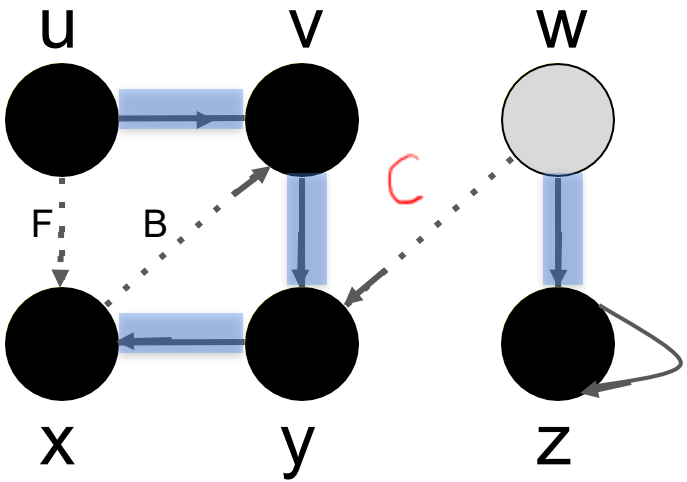
DFS Example



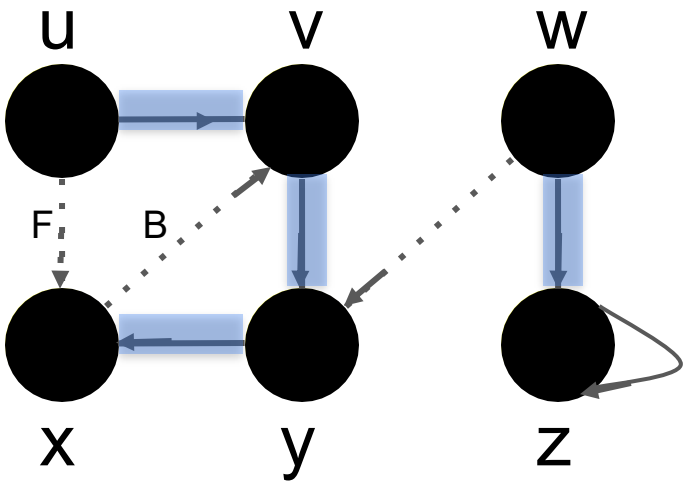
DFS Example



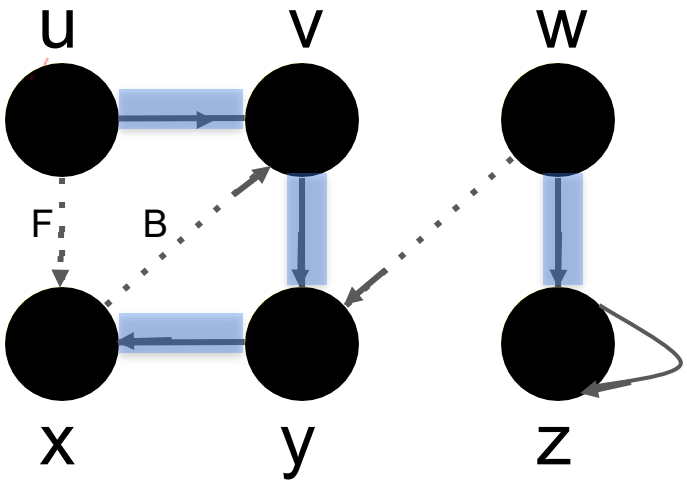
DFS Example

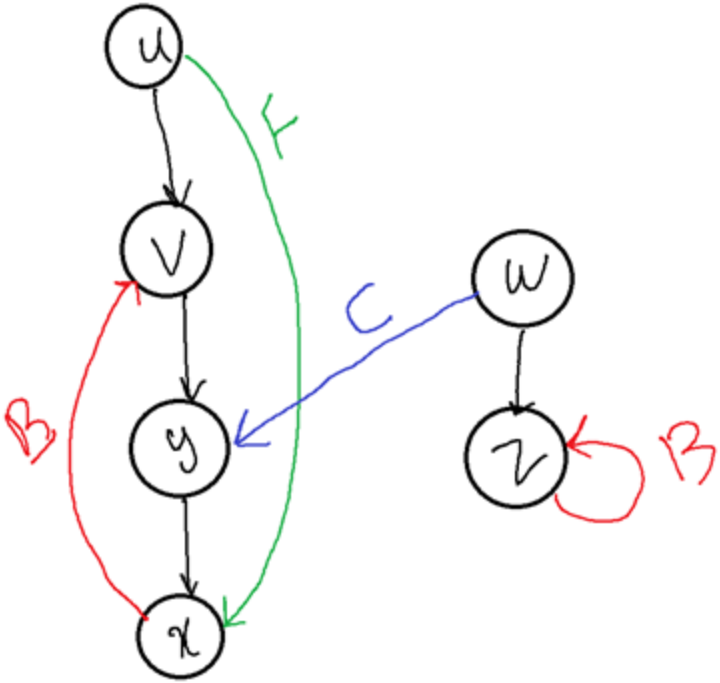


DFS Example



DFS Example





The same graph on the left: re-drawn to specify different classes of edges: B: Backward, F:Forward, C:Cross, and tree edges

DFS: Classification of edges: directed graph

- When we first explore an edge (u, v), the color of vertex v determine edge class
- WHITE: a tree edge
- GRAY: a back edge
- BLACK: a forward or cross edge
  - ?????

Discovery and finishing time

- We modify the algorithm to record the time when a vertex is first visited and the time when its exploring is finished
- These information will be very useful in design and analysis of algorithms

DFS: Enhanced Version

```
time =0
DFS(G)
      for each vertex u in G:
           Color[u] = WHITE
           Pred[u] = NULL
      time =0
      for each vertex u in G:
          if color[u] == WHITE
              DFS(G, u)
DFS(G, u)
     time = time+1
     discover[u] = time
     color[u] = GRAY
     for each neighbor v of u
          if (color[v] == WHITE)
              pred[v] = u
             DFS(G, v)
      color[u] = BLACK
      time = time + 1
     finish[u] = time
```

Enhanced DFS

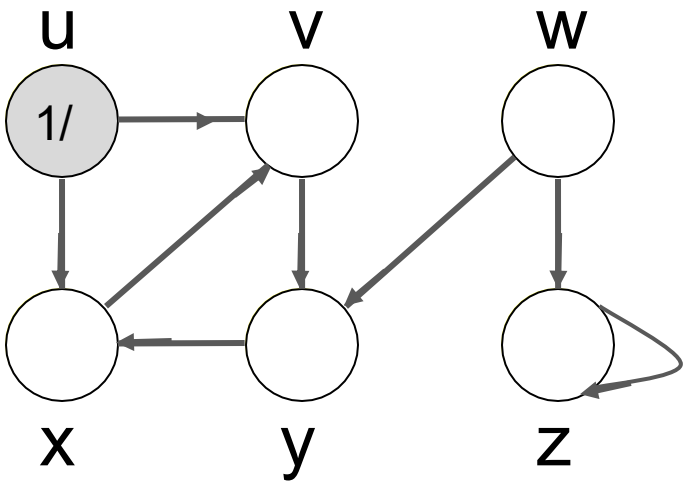
- keeps track of whether a node is undiscovered, discovered, fully explored.
- keeps track of the time it started and finished with a vertex

Output:

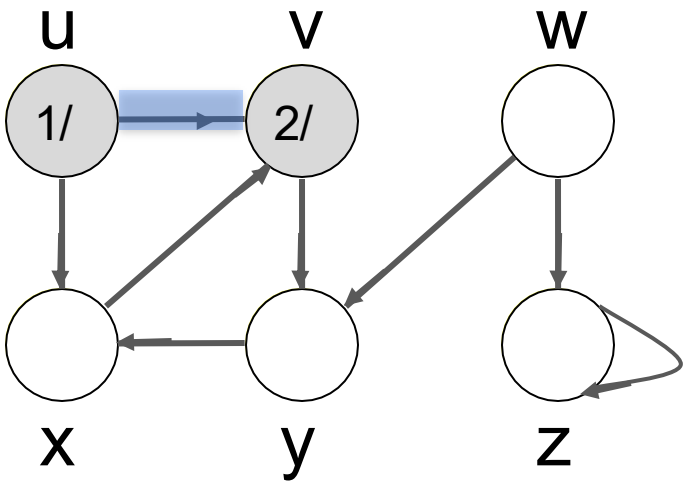
- a depth-first forest
- Timestamp each vertex:
  - d[u], when vertex u was discovered
  - f[u]: when u was fully explored (all its neighbors are discovered)

DFS Example

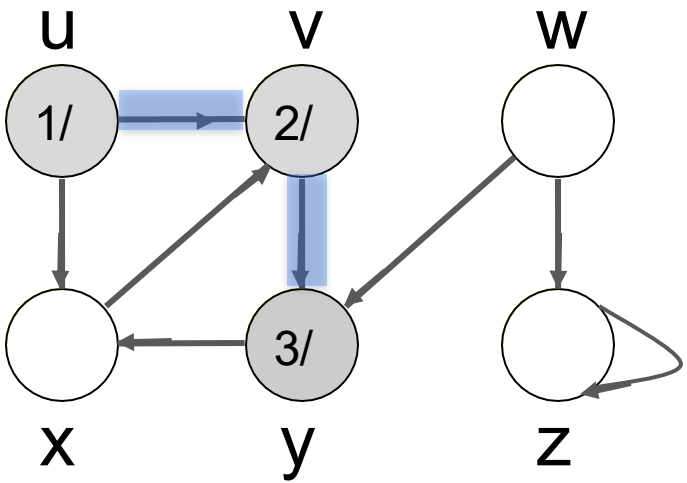
discovery time / finish time



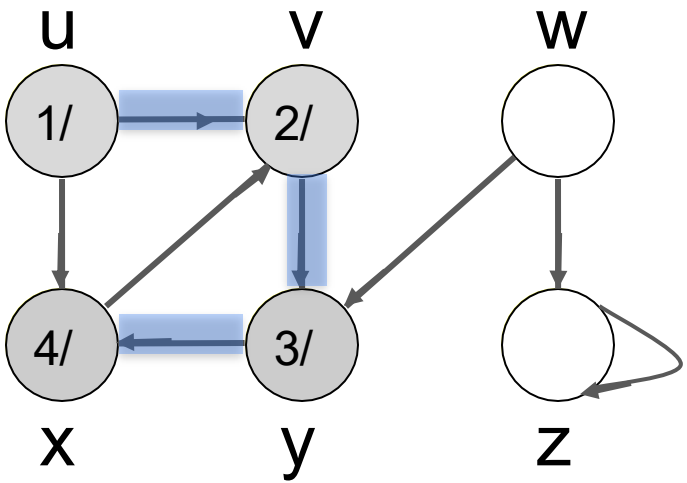
DFS Example



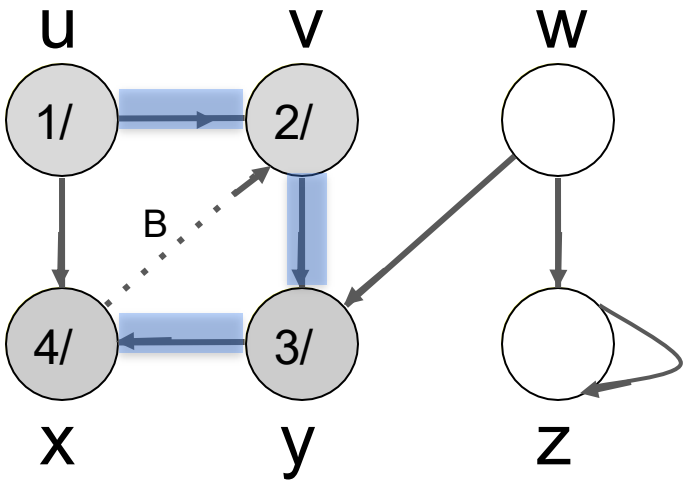
DFS Example



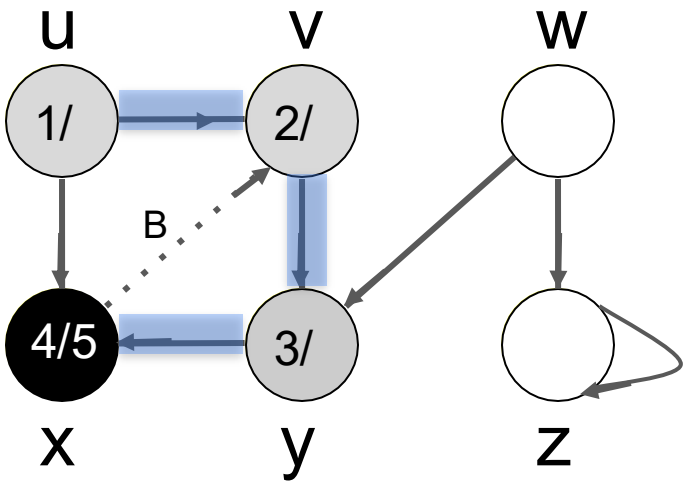
DFS Example



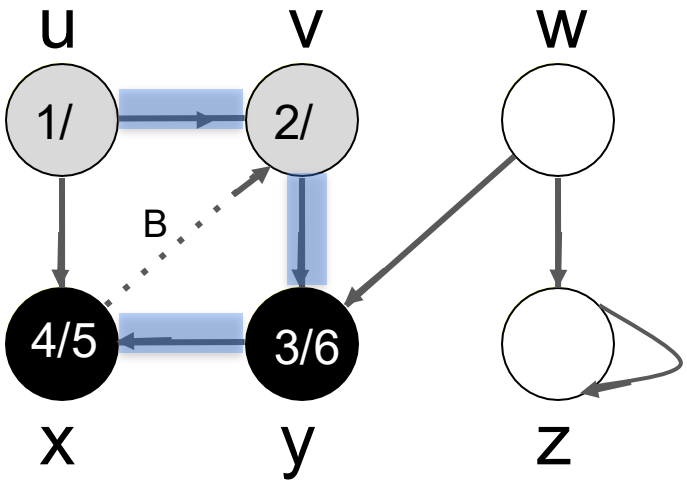
DFS Example



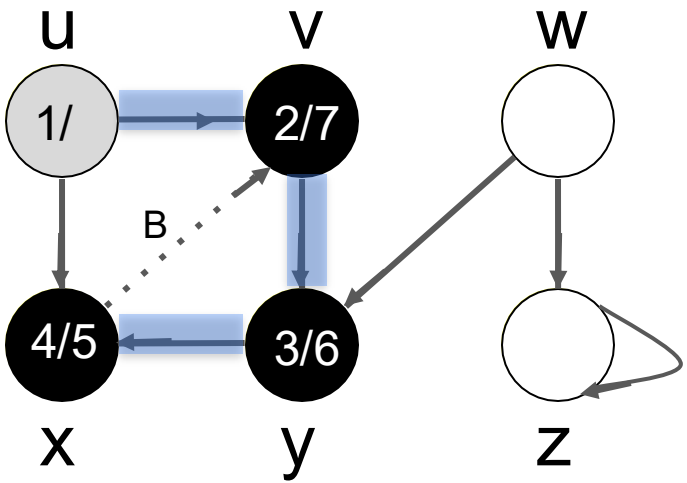
DFS Example



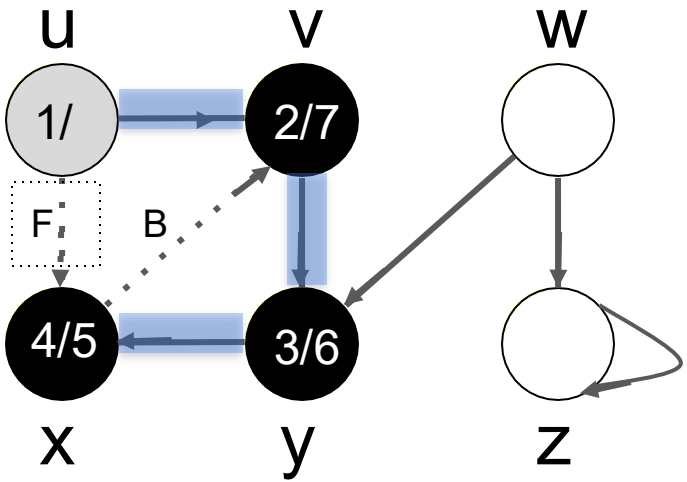
DFS Example



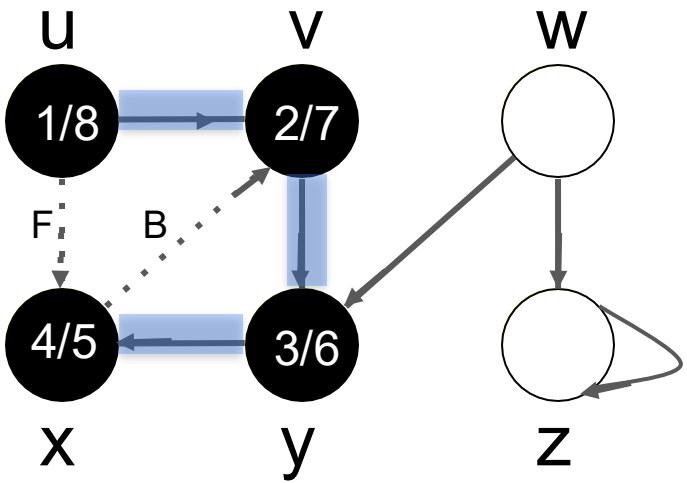
DFS Example



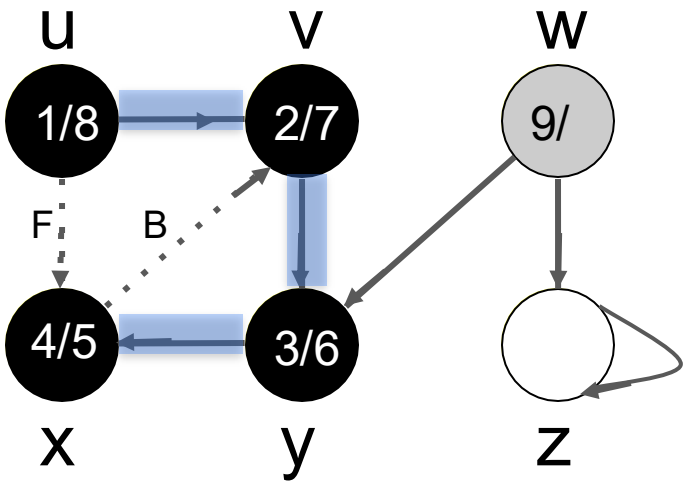
DFS Example



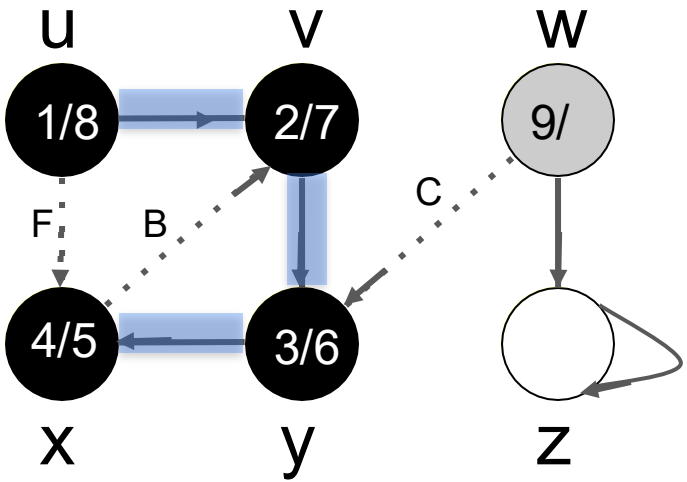
DFS Example



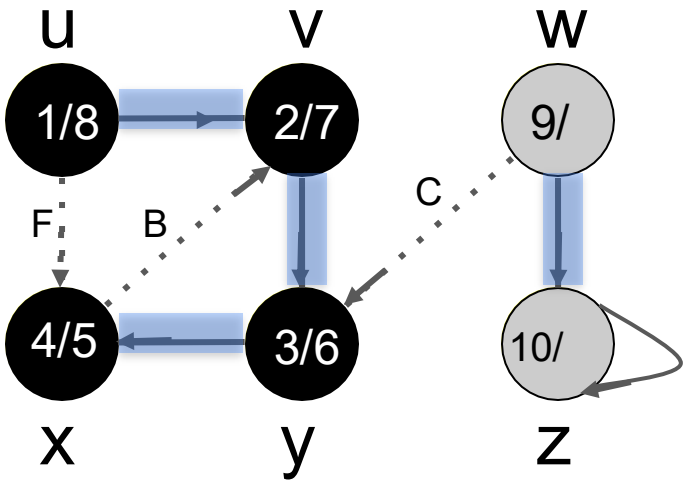
DFS Example



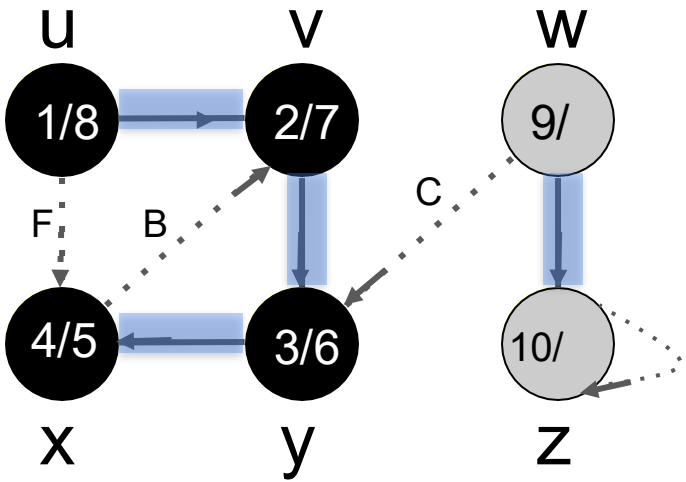
DFS Example



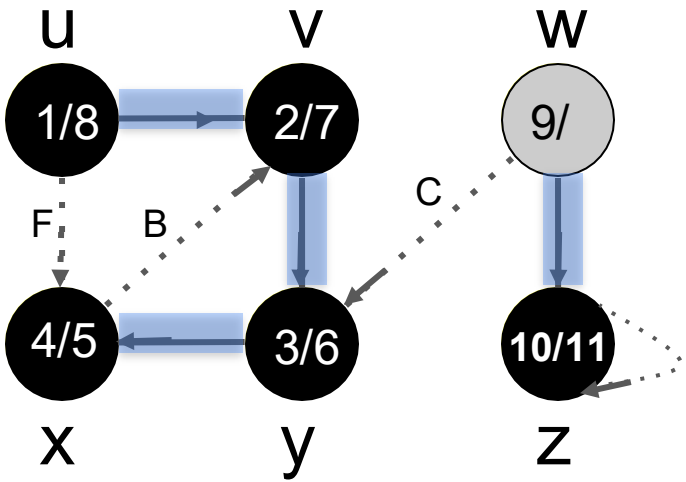
DFS Example



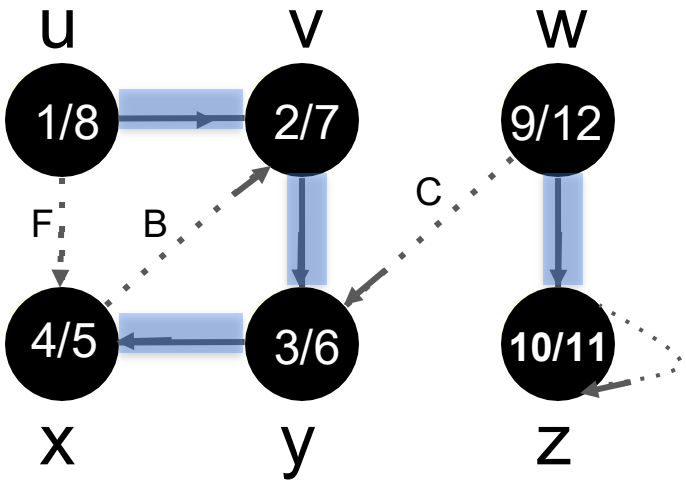
DFS Example



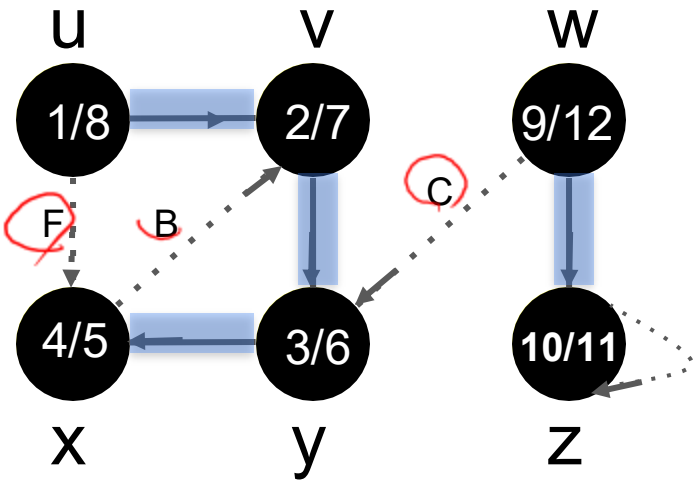
DFS Example

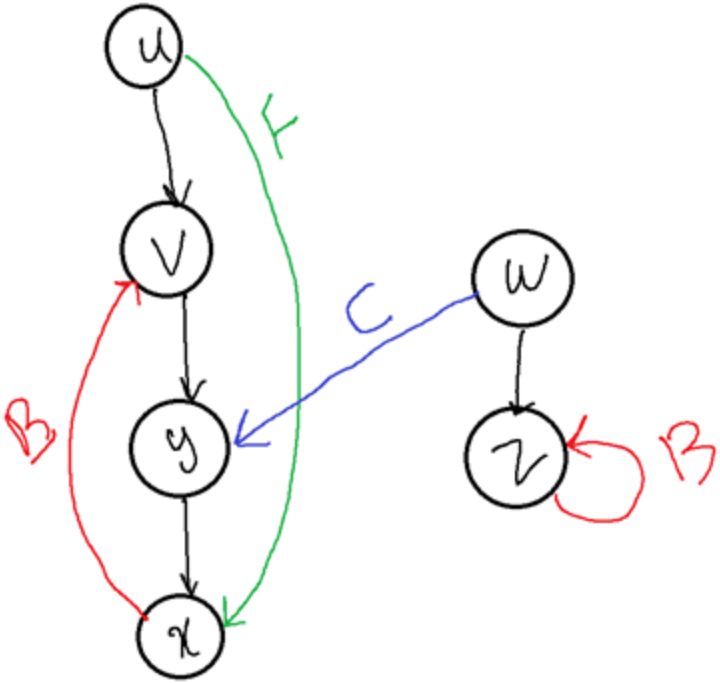


DFS Example



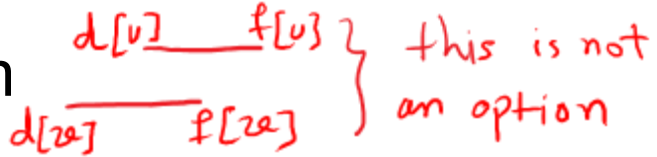
DFS Example



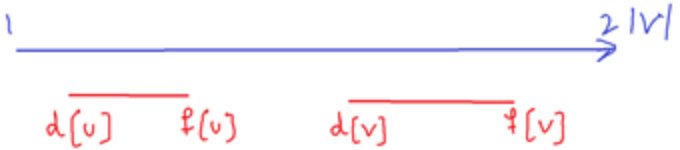


The same graph on the left: re-drawn to specify different classes of edges: B: Backward, F:Forward, C:Cross, and tree edges

DFS properties: Parenthesis theorem



- In any DFS of a graph G, for any two vertices **u** and **v**, the intervals [d[u], f[u]] anad [d[v], f[v]] are either nested or disjoint
  - If two intervals overlap, then one is nested within the other: no partial overlap
  - Vertex corresponding to smaller interval is a descendant of the vertex corresponding to the larger one



Type of edge (u,v)

Cross edge



Tree/forward edge:
Vertex v is a descendant of u

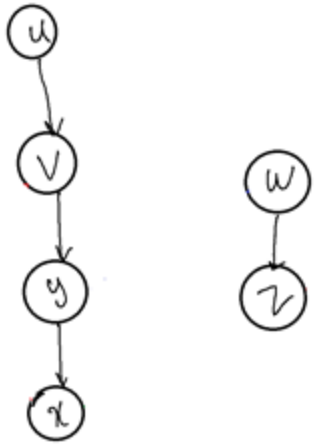


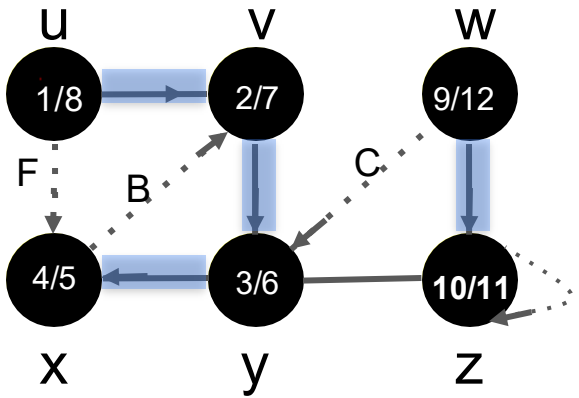
Back edge

DFS: Classification of edges: directed graph

- When we first explore an edge (u, v), the color of vertex v determine edge class
- WHITE: a tree edge
- GRAY: a back edge
- BLACK: a forward or cross edge
  - d[u] < d[v]
    - Forward edge
  - d[u] > d[v]
    - Cross edge

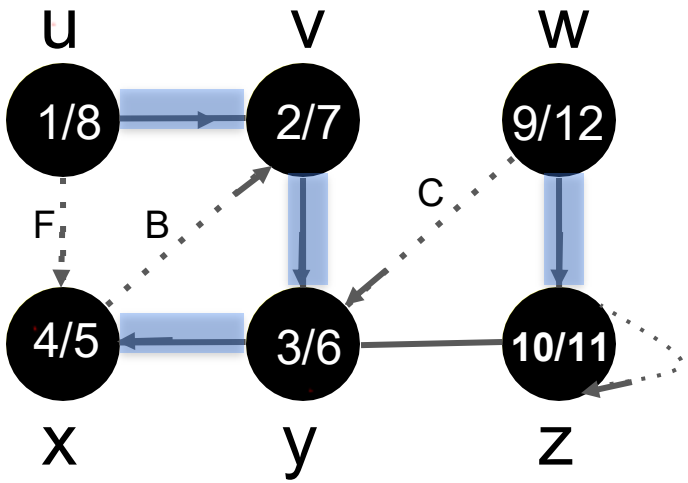
DFS: Edge types and discovery/finish time: Example





DFS: Classification of edges: undirected graph

- In a DFS of an undirected graph G, every edge of the tree:
  - Tree edge
  - Back edge
- There are no cross or forward edges



DFS: Classification of edges: undirected graph

- In a DFS of an undirected graph G, every edge of the tree:
  - Tree edge
  - Back edge
- There are no cross or forward edges
- Proof. Let (u, v) be an edge in G and assume WLOG d[u] < d[v]
  - d[u] < d[v]
    - u becomes gray first, vertex v is finished before u
    - Once u becomes grays there will be two possibilities
      - The neighbor v is discovered → (u,v) becomes a tree edge
      - Some other neighbor w will be discovered and that neighbor discovers v → (v, u) becomes a back edge

learning proof is optional

DFS vs. BFS

- Similarities
  - Both BFS and DFS could be used to
    - Check graph connectivity
    - Find the connected components containing s
      - Both start at an arbitrary node and explores the whole connected component
    - Check s-t connectivity
  - Both BFS(G, s) and DFS(G, s) generate a tree rooted at s
- Differences
  - BFS(G, s) finds the shortest path from s to all vertices reachable from s, but DFS(G, s) does not
  - BFS is implemented using a queue, but not DFS